Milestone 2 — Multi-Object Detection

Study Group 9 · 73 object classes (OBJ001–OBJ073) · YOLOv8s

1. Objective and data

The goal is to detect every target object in a multi-object image and return its **Object ID** and **bounding-box location**. We use all 73 classes (OBJ001–OBJ073); YOLO class indices 0–72 map to these IDs in order. Multi-object images are built by concatenating the 224×224 single-object photos from Milestone 1 into grids.

2. Grid image construction

Each multi-object image places 224×224 single-object photos side by side into a fixed grid; every cell holds one object, and that object's tight box is mapped into the composite. We built 714 images per grid size:

Grid	Objects / image	Image size (px)	Images
2×2	4	448 × 448	714
3×3	9	672 × 672	714
4×4	16	896 × 896	714
5×4	20	1120 × 896	714
5×5	25	1120 × 1120	714
Total	—	—	3,570

Dataset: 3,570 images (train 2,500 / val 535 / test 535), 52,836 object boxes. Classes are balanced (719–728 occurrences each) with shuffled decks. **Augmentation:** each tile is horizontally mirrored with probability 0.5 (box mirrored too) and ±6% brightness/contrast jitter is applied with probability 0.45; YOLO adds mosaic, HSV, translation, scale and flip online at train time. Seed 20261013.

Source separation across splits. The Milestone-1 photo split is kept before any compositing: training grids draw tiles only from training photos, and validation/test grids only from their own pools. We verified that all 6,749 distinct source photos belong to exactly one split (0 sources cross train/val/test).

3. Labels and checks

The shared dataset provides only class labels, so each single-object photo was given one tight box using an open-vocabulary detector (**YOLOE-26**) for proposals, scored by an EfficientNet-B0 classifier and DINOv2 crop-consistency across photos of the same class, with a manual review pass; ambiguous photos got a second text-prompted pass. We checked a sample — a review queue of high-risk cases plus random spot checks — and compared the prompt-free vs text-prompted boxes to catch mistakes.

Known dataset issues handled: exact duplicate photos were de-duplicated *before* splitting (no copy crosses splits); the non-224 image in OBJ003 was resized; **OBJ054_017 was excluded** (not a photo of the object). Of 7,000 object photos, **251 were left out (3.6%**, under the 20% limit): 249 high-risk cases not manually reviewed and 2 with no visible target. All 370 background-only photos were kept.

4. Model and training

We fine-tuned COCO-pretrained **YOLOv8s** with a 73-class detection head (Ultralytics). Local Apple-MPS training hit a PyTorch backward-pass bug, so training was run on a **Kaggle T4 GPU**. Settings: input **896 px** (set close to the large grids so objects are not shrunk too far), batch 16, 50 epochs with patience 10 (early-stopped at epoch 34; best checkpoint epoch 24), AdamW (auto), cosine LR, seed 20261003.

5. Results

Evaluated on the 535 held-out test images (7,918 object boxes):

Metric	Test value
mAP@0.5	0.867
Precision	0.878
Recall	0.851
mAP@0.5:0.95	0.770

Validation at the selected checkpoint was comparable (mAP@0.5 = 0.860, precision 0.877, recall 0.837). Figure 1 shows example detections on test grids; Figure 2 shows the training and validation curves.

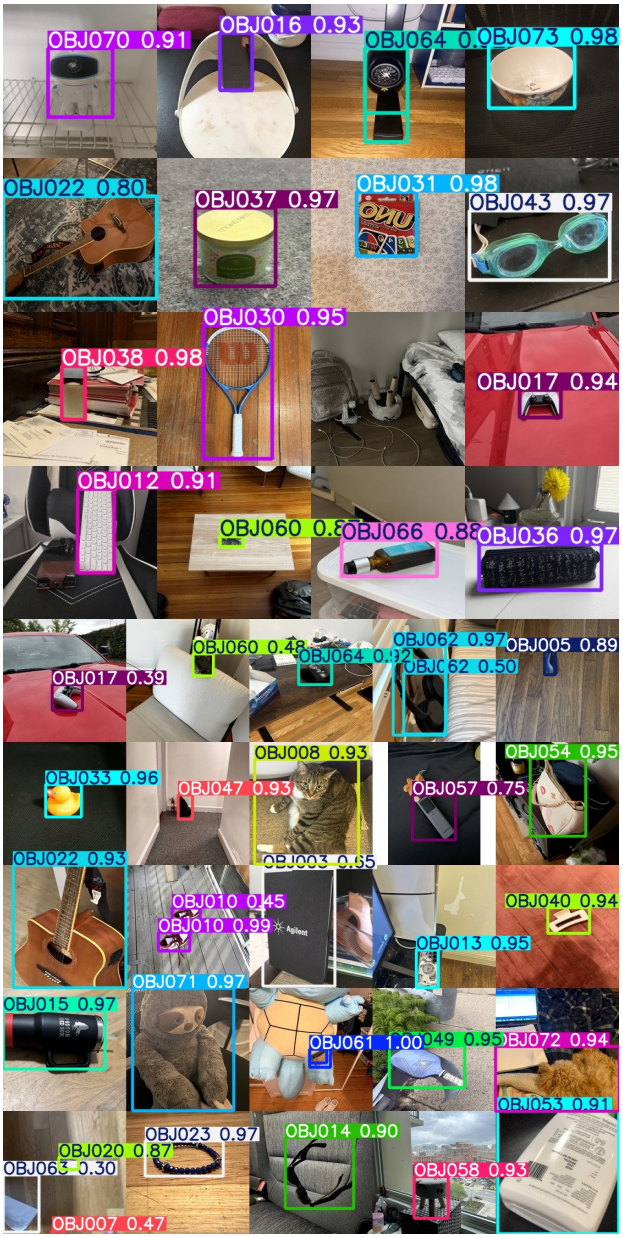


Figure 1. Example detections on held-out test grids. Each red label is the predicted Object ID and confidence; boxes give location.

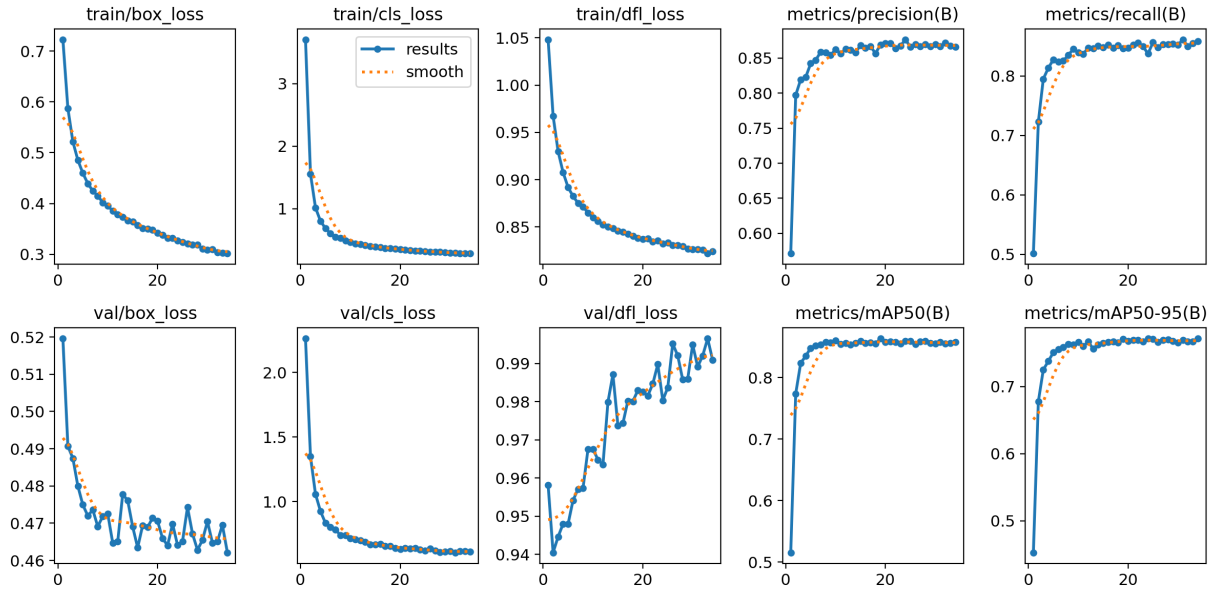


Figure 2. YOLOv8s training/validation curves (losses and val precision, recall, mAP) over 34 epochs.

6. Reproducibility

Code: **https://github.com/Sneha0504/ie7615-object-identification**. The datasets and weights are large and are not committed. The grid dataset is regenerated by `generate_grid_dataset.py` from the single-object labels, and training is reproduced with `kaggle_train_grids.py` on a GPU.